

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ, МЕХАНИКИ И ОПТИКИ
ФАКУЛЬТЕТ ИНФОКОММУНИКАЦИОННЫХ ТЕХНОЛОГИЙ

Отчет по лабораторной работе №1
по курсу «Алгоритмы и структуры данных»
Тема: Сортировка вставками, выбором, пузырьковая.
Вариант 20

Выполнил:
Степанова Е.А.
К3141

Проверил:
Афанасьев А.В.

Санкт-Петербург
2024г.

Содержание отчета

Содержание отчета	2
Задачи по варианту	3
<u>Задача №1. Сортировка вставкой [N баллов]</u>	3
<u>Задача №3. Сортировка вставкой по убыванию [N баллов]</u>	9
<u>Задача №4. Линейный поиск [N баллов]</u>	15
<u>Задача №5. Сортировка выбором [N баллов]</u>	20
<u>Задача №6. Пузырьковая сортировка [N баллов]</u>	25
<u>Задача №8. Секретарь Своп [N баллов]</u>	30
<u>Задача №9. Сложение двоичных чисел [N баллов]</u>	34
 Вывод	 38

Задачи по варианту

Задача №1. Сортировка вставкой [N баллов]

Используя код процедуры Insertion-sort, напишите программу и проверьте сортировку массива $A = \{31, 41, 59, 26, 41, 58\}$.

- **Формат входного файла (input.txt).** В первой строке входного файла содержится число n ($1 \leq n \leq 10^3$) — число элементов в массиве. Во второй строке находятся n различных целых чисел, по модулю не превосходящих 10^9 .
- **Формат выходного файла (output.txt).** Одна строка выходного файла с отсортированным массивом. Между любыми двумя числами должен стоять ровно один пробел.
- Ограничение по времени. 2сек.
- Ограничение по памяти. 256 мб.

Выберите любой набор данных, подходящих по формату, и протестируйте алгоритм.

```
import time
from memory_profiler import profile

t_start = time.perf_counter()

@profile()
def insertion_sort(a, n):
    for i in range(1, n):
        for j in range(i, 0, -1):
            if a[j] < a[j - 1]:
                a[j], a[j - 1] = a[j - 1], a[j]
            else:
                break

    return a

if __name__ == '__main__':
    f1 = open('../txtf/input.txt', 'r')
```

```

n = int(f1.readline())
if (1 <= n <= 10 ** 3):
    a = list(map(int, f1.readline().split()))
    f2 = open('../txtf/output.txt', 'w')
    f2.write(str(insertion_sort(a, n)))
    f2.close()

else:
    print('Error.Try again')
    f1.close()
t_start = time.perf_counter()
print("Время работы: %s секунд" %
(time.perf_counter() - t_start))

```

```

import time
from memory_profiler import profile

```

```

t_start = time.perf_counter()

```

```

@profile()
def insertion_sort(a, n):
    for i in range(1, n):
        for j in range(i, 0, -1):
            if a[j] < a[j - 1]:
                a[j], a[j - 1] = a[j - 1], a[j]
            else:
                break

    return a

```

```

if __name__ == '__main__':
    f1 = open('../txtf/input.txt', 'r')
    n = int(f1.readline())
    if (1 <= n <= 10 ** 3):
        a = list(map(int, f1.readline().split()))
        f2 = open('../txtf/output.txt', 'w')

```

```

f2.write(str(insertion_sort(a, n)))
f2.close()

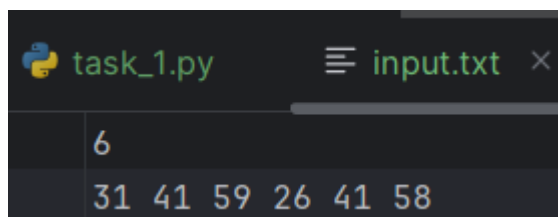
else:
    print('Error.Try again')
    f1.close()
t_start = time.perf_counter()
print("Время работы: %s секунд" % (time.perf_counter() - t_start))

```

Описание кода.

Создаю отдельную функцию для сортировки вставкой. В ней задаю два цикла, которые мне помогут проходиться по элементам внутри списка. Во втором цикле проверяю, что если элемент, индекс которого больше, меньше по значению элемента, индекс которого меньше, то меняю эти элементы местами, если это не так, то прекращаю второй цикл на данной итерации. После прохода всего первого цикла и получения значений, возвращаю отсортированный список.

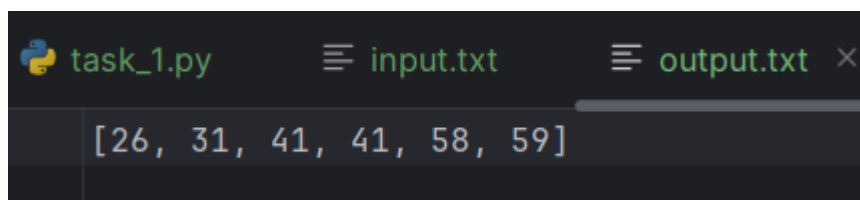
В основном коде открываю файл для чтения, считываю оттуда все данные, проверяю `n` на условие, если не выполняется, то сообщая пользователю об ошибке, если выполняется, то считываю из файла значения для списка и присваиваю их ему. Открываю файл для ответа и записываю туда результат функции сортировки вставкой.



```

task_1.py  input.txt x
6
31 41 59 26 41 58

```



```

task_1.py  input.txt  output.txt x
[26, 31, 41, 41, 58, 59]

```


[illegible]

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	1.600012183189392e-06 секунд	22.4 MiB
Пример из задачи	1.700012944638729e-06 секунд	22.4 MiB
Верхняя граница диапазона значений входных данных из текста задачи	2.400018274784088e-06 секунд	22.4 MiB

Вывод:

Сортировка вставками является эффективной для небольшого количества значений, тк при их увеличении увеличивается и время работы.

Задача №3. Сортировка вставкой по убыванию[N баллов]

3 задача. Сортировка вставкой по убыванию

Перепишите процедуру Insertion-sort для сортировки в невозрастающем порядке вместо неубывающего с использованием процедуры **Swap**.

Формат входного и выходного файла и ограничения - как в задаче 1.

Подумайте, можно ли переписать алгоритм сортировки вставкой с использованием рекурсии?

```
import time
from memory_profiler import profile

t_start = time.perf_counter()
```

```

@profile()
def insertion_sort(a, n):
    res = []
    for i in range(1, n):
        for j in range(i, 0, -1):
            if a[j] > a[j - 1]:
                a[j], a[j - 1] = a[j - 1], a[j]
            else:
                break

    return a

if __name__ == '__main__':
    f1 = open('../txtf/input.txt', 'r')
    n = int(f1.readline())
    if (1 <= n <= 10 ** 3):
        a = list(map(int, f1.readline().split()))
        f2 = open('../txtf/output.txt', 'w')
        f2.write(str(insertion_sort(a, n)))
        f2.close()

    else:
        print('Error.Try again')
        f1.close()
t_start = time.perf_counter()
print("Время работы: %s секунд" %
      (time.perf_counter() - t_start))

```

```

import time
from memory_profiler import profile

```

```

t_start = time.perf_counter()

```

```

@profile()
def insertion_sort(a, n):
    res = []

```

```

for i in range(1, n):
    for j in range(i, 0, -1):
        if a[j] > a[j - 1]:
            a[j], a[j - 1] = a[j - 1], a[j]
        else:
            break

return a

if __name__ == '__main__':
    f1 = open('./txtf/input.txt', 'r')
    n = int(f1.readline())
    if (1 <= n <= 10 ** 3):
        a = list(map(int, f1.readline().split()))
        f2 = open('./txtf/output.txt', 'w')
        f2.write(str(insertion_sort(a, n)))
        f2.close()

    else:
        print('Error.Try again')
        f1.close()
t_start = time.perf_counter()
print("Время работы: %s секунд" % (time.perf_counter() - t_start))

```

Описание кода.

Создаю отдельную функцию для сортировки вставкой. В ней задаю два цикла, которые мне помогут проходиться по элементам внутри списка. Во втором цикле проверяю, что если элемент, индекс которого больше, больше по значению элемента, индекс которого меньше, то меняю эти элементы местами, если это не так, то прекращаю второй цикл на данной итерации. После прохода всего первого цикла и получения значений, возвращаю отсортированный список.

В основном коде открываю файл для чтения, считываю оттуда все данные, проверяю `n` на условие, если не выполняется, то сообщая пользователю об ошибке, если выполняется, то считываю из файла значения для списка и присваиваю их ему. Открываю файл для ответа и записываю туда результат функции сортировки вставкой.

[illegible]

Выходные данные:

[illegible]

Task_3\...\input.txt ×		task_3.py	Task_3\...\output.txt ×	
1	1	✓	1	[1]
2	1			

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	1.100008375942707e-06 секунд	22.3 MiB
Пример из задачи	1.200009137392044e-06 секунд	22.3 MiB
Верхняя граница диапазона значений входных данных из текста задачи	7.00005330145359e-07 секунд	22.4 MiB

Вывод:

Сортировка вставками по убыванию является эффективной для небольшого количества значений, тк при их увеличении увеличивается и время работы и затраченная память.

Задача №4. Линейный поиск[N баллов]

4 задача. Линейный поиск

Рассмотрим задачу поиска.

- **Формат входного файла.** Последовательность из n чисел $A = a_1, a_2, \dots, a_n$ в первой строке, числа разделены пробелом, и значение V во второй строке. Ограничения: $0 \leq n \leq 10^3$, $-10^3 \leq a_i, V \leq 10^3$
- **Формат выходного файла.** Одно число - индекс i , такой, что $V = A[i]$, или значение -1 , если V в отсутствует.
- Напишите код линейного поиска, при работе которого выполняется сканирование последовательности в поисках значения V .
- Если число встречается несколько раз, то выведите, сколько раз встречается число и все индексы i через запятую.
- Дополнительно: попробуйте найти свинью, как в лекции. Используйте во входном файле последовательность слов из лекции, и найдите соответствующий индекс.

```
import time
from memory_profiler import profile

t_start = time.perf_counter()

@profile()
def lin_search(a, n, v):
    res = []
    k = 0
    for i in range(n):
        if a[i] == v:
            k += 1
            res.append(i)
    return k, res

if __name__ == '__main__':
    f1 = open('../txtf/input.txt', 'r')
    f2 = open('../txtf/output.txt', 'w')
    n = int(f1.readline())
```

```

a = list(map(int, f1.readline().split()))
v = int(f1.readline())
if (0 <= n <= 10 ** 3) and (-10 ** 3 <= min(a),
max(a), v <= 10 ** 3):
    k, res = lin_search(a, n, v)
    if k > 1:
        f2.write(f'{str(k)}\n')
        res = [*map(str, res)]
        for i in range(len(res) - 1):
            f2.write(f'{res[i]}, ')
        f2.write(f'{res[len(res) - 1]}\n')

    elif k == 1:
        f2.write(str(a[0]))
    else:
        f2.write(str(-1))
    f2.close()

else:
    print('Error.Try again')
f1.close()

t_start = time.perf_counter()
print("Время работы: %s секунд" %
(time.perf_counter() - t_start))

```

```

import time
from memory_profiler import profile

```

```

t_start = time.perf_counter()

```

```

@profile()
def lin_search(a, n, v):
    res = []
    k = 0
    for i in range(n):
        if a[i] == v:
            k += 1
            res.append(i)

```

```

return k, res

if __name__ == '__main__':
    f1 = open('./txtf/input.txt', 'r')
    f2 = open('./txtf/output.txt', 'w')
    n = int(f1.readline())
    a = list(map(int, f1.readline().split()))
    v = int(f1.readline())
    if (0 <= n <= 10 ** 3) and (-10 ** 3 <= min(a), max(a), v <= 10 ** 3):
        k, res = lin_search(a, n, v)
        if k > 1:
            f2.write(f'{str(k)}\n')
            res = [*map(str, res)]
            for i in range(len(res) - 1):
                f2.write(f'{res[i]}, ')
            f2.write(f'{res[len(res) - 1]}\n')

        elif k == 1:
            f2.write(str(a[0]))
        else:
            f2.write(str(-1))
        f2.close()

    else:
        print('Error.Try again')
    f1.close()

t_start = time.perf_counter()
print("Время работы: %s секунд" % (time.perf_counter() - t_start))

```

Описание решения.

Завожу отдельную функцию, в нее подаю список, длину списка и искомое значение. Так как это линейный поиск, то суть его заключается в том, чтобы пройти циклом по всем элементам и сравнивать их с искомым,

если нашелся, в счетчик увеличиваю на 1, а индекс найденного элемента записываю в список `res`, так прохожусь до конца, возвращаю счетчик и список.

В основном коде, открываю файлы для записи и чтения, считываю данные, проверяю условие, если условие выполняется, то присваиваю переменным `k` и `res` результат выполнения функции `lin_search`. По условию, если $k > 1$, то вывожу количество таких элементов и через запятую их индексы. Если $k == 1$, то вывожу индекс, случаях вывожу -1, закрываю файлы.

task_4.pyinput.txtTask_4\...\output.txt

1	4	✓	1	-1
2	3 2 3 2			
3	1			

task_4.pyinput.txtTask_4\...\output.txt

1	1	✓	1	1
2	1			
3	1			

task_4.pyoutput.txtinput.txt

1	4	✓	1	15	✓
2	7, 8, 11, 12		2	1 2 3 4 5 6 6 8 8 9 1 8 8 9 1	
			3	8	

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	6.00004568696022e-07 секунд	22.4 MiB
Пример из задачи	7.00005330145359e-07	22.4 MiB

	секунд	
Пример из задачи	8.00006091594696e-07 секунд	22.4 MiB
Верхняя граница диапазона значений входных данных из текста задачи	6.00004568696022e-07 секунд	22.5 MiB

Вывод:

При увеличении входных данных падает эффективность алгоритма.

Задача №5. Сортировка выбором [N баллов]

5 задача. Сортировка выбором.

Рассмотрим сортировку элементов массива, которая выполняется следующим образом. Сначала определяется наименьший элемент массива, который ставится на место элемента $A[1]$. Затем производится поиск второго наименьшего элемента массива A , который ставится на место элемента $A[2]$. Этот процесс продолжается для первых $n - 1$ элементов массива A .

Напишите код этого алгоритма, также известного как сортировка выбором (selection sort). Определите время сортировки выбором в наихудшем случае и в среднем случае и сравните его со временем сортировки вставкой.

Формат входного и выходного файла и ограничения - как в задаче 1.

3

```
import time
from memory_profiler import profile

t_start = time.perf_counter()

@profile()
def selection_sort(a, n):
    for i in range(n - 1):
        min_elem = i
        for j in range(i + 1, n):
            if a[j] < a[min_elem]:
                min_elem = j
        a[i], a[min_elem] = a[min_elem], a[i]
```

```

        return a

if __name__ == '__main__':
    f1 = open('../txtf/input_Task_5.txt', 'r')
    n = int(f1.readline())
    a = list(map(int, f1.readline().split()))
    f2 = open('../txtf/output_Task_5.txt', 'w')
    f2.write(str(selection_sort(a, n)))
    f2.close()

    f1.close()
t_start = time.perf_counter()
print("Время работы: %s секунд" %
      (time.perf_counter() - t_start))

```

```

import time
from memory_profiler import profile

t_start = time.perf_counter()

@profile()
def selection_sort(a, n):
    for i in range(n - 1):
        min_elem = i
        for j in range(i + 1, n):
            if a[j] < a[min_elem]:
                min_elem = j
        a[i], a[min_elem] = a[min_elem], a[i]

    return a

```

```

if __name__ == '__main__':
    f1 = open('../txtf/input_Task_5.txt', 'r')

```

```
n = int(f1.readline())
a = list(map(int, f1.readline().split()))
f2 = open('./txtf/output_Task_5.txt', 'w')
f2.write(str(selection_sort(a, n)))
f2.close()

f1.close()
t_start = time.perf_counter()
print("Время работы: %s секунд" % (time.perf_counter() - t_start))
```

Описание решения.

Смысл данного алгоритма в том, чтобы найти минимальный элемент, потом поставить его в начало списка, дальше снова найти минимальный элемент(переставленный первый элемент в рассмотрение не берем) и снова ставим его в начало списка, но уже на второе место и так далее.

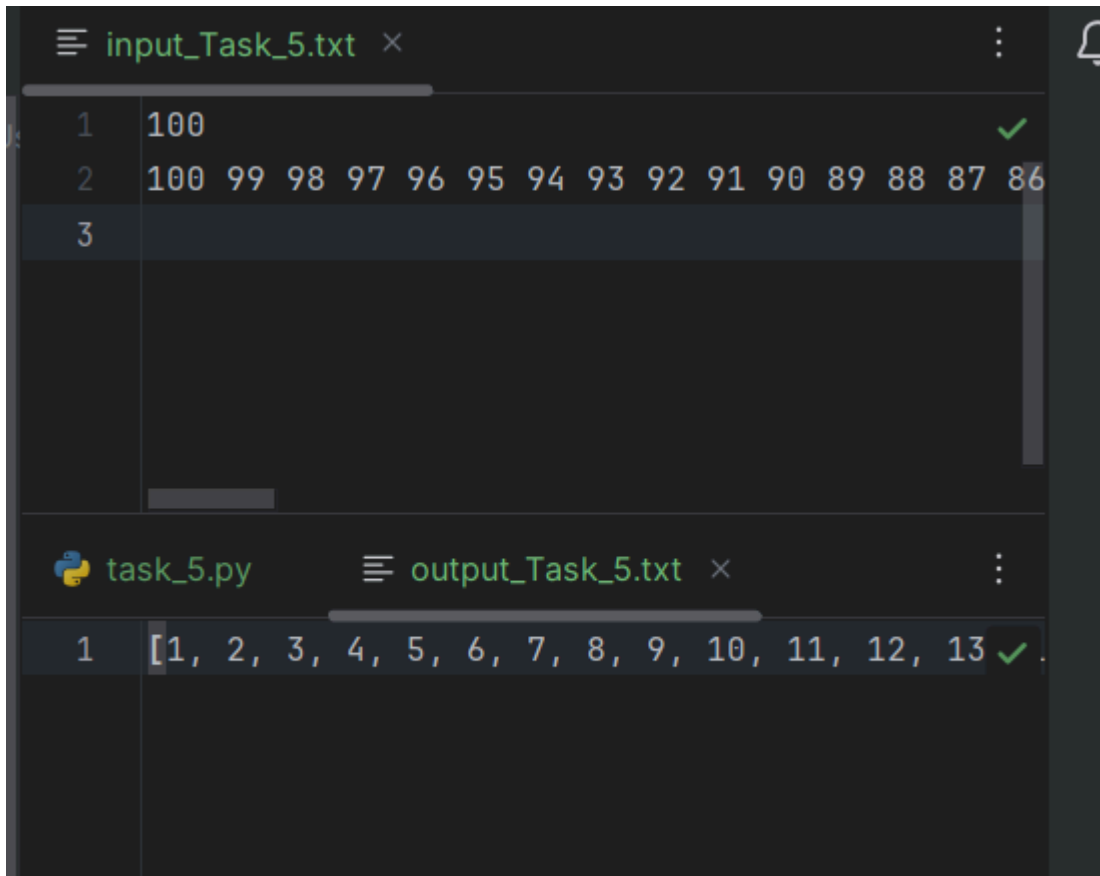
Поэтому я создала отдельную функцию `selection_sort`, в ней задаю цикл: первый отвечает за “точку отсчета движения по циклу”(рассматриваем элементы со второго, третьего или четвертого элемента и тд.) Потом говорю, что беру за индекс минимального элемента `i` от которого и будем начинать двигаться. Задаю второй цикл, он отвечает за проход по элементам, начиная с точки отсчета до конца списка. Внутри него проверяю, если найденный элемент меньше чем элемент от минимального индекса, то присваиваю минимальному индексу значение `j`.

После окончательного нахождения минимального индекса, меняю местами `a[i]` и `a[min_elem]`. Возвращаю отсортированный список.

В основном коде открываю файл для чтения, присваиваю `n` значение, присваиваю списку значения и открываю файл для записи, записываю в файл для ответа значение функции `selection_sort`, закрываю файлы.

```
input_Task_5.txt ×
1 6
2 31 41 59 26 41 58

task_5.py  output_Task_5.txt ×
1 [26, 31, 41, 41, 58, 59]
```



Входные

```
100
100 99 98 97 96 95 94 93 92 91 90 89 88 87 86 85 84
83 82 81 80 79 78 77 76 75 74 73 72 71 70 69 68 67
66 65 64 63 62 61 60 59 58 57 56 55 54 53 52 51 50
49 48 47 46 45 44 43 42 41 40 39 38 37 36 35 34 33
32 31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16
15 14 13 12 11 10 9 8 7 6 5 4 3 2 1
```

Выходные

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15,
16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28,
29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41,
42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54,
55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67,
68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80,
81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93,
94, 95, 96, 97, 98, 99, 100]
```

```
input_Task_5.txt x
1 1
2 1

task_5.py output_Task_5.txt x
1 [1]
```

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	6.00004568696022e-07 секунд	22.3 MiB
Пример из задачи	7.00005330145359e-07 секунд	22.4 MiB
Верхняя граница диапазона значений входных данных из текста задачи	7.00005330145359e-07 секунд	22.3 MiB

Вывод:

При увеличении входных данных увеличивается и затраченное время, если сравнивать этот алгоритм с алгоритмом сортировки выбором, то сортировка вставкой более эффективна, тк вместо 6 секунд она тратит в 2 раза меньше, а затраты памяти у них одинаковые.

Задача №6. Пузырьковая сортировка [N баллов]

6 задача. Пузырьковая сортировка

Пузырьковая сортировка представляет собой популярный, но не очень эффективный алгоритм сортировки. В его основе лежит многократная перестановка соседних элементов, нарушающих порядок сортировки. Вот псевдокод этой сортировки:

```
Bubble_Sort(A):  
  for i = 1 to A.length - 1  
    for j = A.length downto i+1  
      if A[j] < A[j-1]  
        поменять A[j] и A[j-1] местами
```

Напишите код на Python и докажите корректность пузырьковой сортировки. Для доказательства корректности процедуры вам необходимо доказать, что она завершается и что $A'[1] \leq A'[2] \leq \dots \leq A'[n]$, где A' - выход процедуры Bubble_Sort, а n - длина массива A .

Определите время пузырьковой сортировки в наихудшем случае и в среднем случае и сравните его со временем сортировки вставкой.

Формат входного и выходного файла и ограничения - как в задаче 1.

```
import time  
from memory_profiler import profile  
  
t_start = time.perf_counter()  
  
@profile()  
def bubble_sort(a):  
    for i in range(1, len(a)):  
        for j in range(len(a) - 1, i + 1, -1):  
            if a[j] < a[j - 1]:  
                a[j], a[j - 1] = a[j - 1], a[j]  
    return a  
  
if __name__ == '__main__':  
    f1 = open('../txtf/input_Task9.txt', 'r')  
    a = list(map(int, f1.readline().split()))  
    f2 = open('../txtf/output_Task9.txt', 'w')  
    ans = [*map(str, bubble_sort(a))]  
    for i in ans:  
        f2.write(f'{i} ')  
    f2.close()  
    f1.close()  
    t_start = time.perf_counter()
```

```
print("Время работы: %s секунд" %  
(time.perf_counter() - t_start))
```

```
import time  
from memory_profiler import profile
```

```
t_start = time.perf_counter()
```

```
@profile()  
def bubble_sort(a):  
    for i in range(1, len(a)):  
        for j in range(len(a) - 1, i + 1, -1):  
            if a[j] < a[j - 1]:  
                a[j], a[j - 1] = a[j - 1], a[j]  
    return a
```

```
if __name__ == '__main__':  
    f1 = open('./txtf/input_Task9.txt', 'r')  
    a = list(map(int, f1.readline().split()))  
    f2 = open('./txtf/output_Task9.txt', 'w')  
    ans = [*map(str, bubble_sort(a))]  
    for i in ans:  
        f2.write(f'{i} ')  
    f2.close()  
    f1.close()  
t_start = time.perf_counter()  
print("Время работы: %s секунд" % (time.perf_counter() - t_start))
```

Описание решения.

Смысл этой сортировки заключается в том, чтобы последовательно сравнивать соседние элементы друг с другом и обменивать их в случае невыполнения условия(смотря как мы сортируем по возрастанию или по убыванию).

В основном коде, открываю файл для чтения, считываю оттуда список `a`, открываю файл для ответа, завожу список `ans`, он повторяет отсортированный список `a`, только я каждому элементу присваиваю строковое значение, тк записывать в файл можно объекты типа `str`.
С помощью цикла пробегаюсь по `ans` записываю его значения в файл2.

```
task_6.py × input_Task9.txt × output_Task9.txt ×
1 2 3 10 6 7 ✓ 1 2 3 6 7 10
```

The screenshot shows a code editor with two files open: `task_6.py` and `input_Task9.txt`. The `input_Task9.txt` file contains the following sequence of numbers: 1 1 1 1 1 1 1 1 1 1 1 1 8 8 1. The `output_Task9.txt` file contains the following sequence of numbers: 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1. Both sequences are followed by a green checkmark, indicating successful execution or completion.

Входные

[illegible]

Вывод:

Данный алгоритм является эффективным для маленького количества значений, тк при их увеличении значительно увеличивается и время выполнения. Если сравнивать этот алгоритм с сортировкой вставкой, то сортировка вставкой опять же намного эффективнее, чем пузырьковая, потому что занимает гораздо меньше времени.

Задача №8. Секретарь Своп [N баллов]

8 задача. Секретарь Своп

Дан массив, состоящий из n целых чисел. Вам необходимо его отсортировать по неубыванию. Но делать это нужно так же, как это делает мистер Своп — то есть, каждое действие должно быть взаимной перестановкой пары элементов. Вам также придется записать все, что Вы делали, в файл, чтобы мистер Своп смог проверить Вашу работу.

- **Формат входного файла (input.txt).** В первой строке входного файла содержится число n ($3 \leq n \leq 5000$) — число элементов в массиве. Во второй строке находятся n целых чисел, по модулю не превосходящих 10^9 . Числа могут совпадать друг с другом.

- **Формат выходного файла (output.txt).** В первых нескольких строках выведите осуществленные Вами операции перестановки элементов. Каждая строка должна иметь следующий формат:

Swap elements at indices X and Y .

Здесь X и Y — различные индексы массива, элементы на которых нужно переставить ($1 \leq X, Y \leq n$). Мистер Своп любит порядок, поэтому сделайте так, чтобы $X < Y$.

После того, как все нужные перестановки выведены, выведите следующую фразу:

No more swaps needed.

- Пример:

input.txt	output.txt
5	Swap elements at indices 1 and 2.
3 1 4 2 2	Swap elements at indices 2 and 4.
	Swap elements at indices 3 and 5.
	No more swaps needed.

```
import time
from memory_profiler import profile
```

```

t_start = time.perf_counter()

@profile()
def mister_swap(a, n, f2):
    for i in range(n - 1):
        min_id = i
        min = a[i]
        for j in range(i + 1, n):
            if a[j] < min:
                min = a[j]
                min_id = j
        if min_id != i:
            a[i], a[min_id] = a[min_id], a[i]
            f2.write(f'Swap elements at indices {i +
1} and {min_id + 1}.\n')

if __name__ == '__main__':
    f1 = open('../txtf/input_Task8.txt', 'r')
    n = int(f1.readline())
    if (3 <= n <= 5 * 10 ** 3):
        a = list(map(int, f1.readline().split()))
        f2 = open('../txtf/output_Task8.txt', 'w')
        mister_swap(a, n, f2)
        f2.write('No more swaps needed.')
        f2.close()

    else:
        print('Error.Try again')
        f1.close()
t_start = time.perf_counter()
print("Время работы: %s секунд" %
(time.perf_counter() - t_start))

```

```

import time
from memory_profiler import profile

```

```
t_start = time.perf_counter()
```

```
@profile()
```

```
def mister_swap(a, n, f2):
```

```
    for i in range(n - 1):
```

```
        min_id = i
```

```
        min = a[i]
```

```
        for j in range(i + 1, n):
```

```
            if a[j] < min:
```

```
                min = a[j]
```

```
                min_id = j
```

```
    if min_id != i:
```

```
        a[i], a[min_id] = a[min_id], a[i]
```

```
        f2.write(f'Swap elements at indices {i + 1} and {min_id + 1}.\n')
```

```
if __name__ == '__main__':
```

```
    f1 = open('./txtf/input_Task8.txt', 'r')
```

```
    n = int(f1.readline())
```

```
    if (3 <= n <= 5 * 10 ** 3):
```

```
        a = list(map(int, f1.readline().split()))
```

```
        f2 = open('./txtf/output_Task8.txt', 'w')
```

```
        mister_swap(a, n, f2)
```

```
        f2.write('No more swaps needed.')
```

```
        f2.close()
```

```
    else:
```

```
        print('Error.Try again')
```

```
    f1.close()
```

```
t_start = time.perf_counter()
```

```
print("Время работы: %s секунд" % (time.perf_counter() - t_start))
```

Описание решения.

Создаю функцию `mister_swap()`, в ней передаю список, длину списка и файл для записи.

Завожу цикл, говорю, пусть минимальный индекс это i , а минимальный элемент тогда $a[i]$.

Прохожусь по второму циклу, проверяю: если текущий элемент меньше чем минимальный, то тогда минимальным элементом будет текущий, аналогично и для индекса. Вне второго цикла делаю проверку, что если мин индекс не равен тому, что я задала изначально, значит “значит нашел элемент который можно переставить” -> делаю перестановку в массиве a (сортирую), а в файл для записи ответа пишу какие элементы (начиная с 1) я переставила.

В основном коде проверяю открываю файл для чтения, считываю данные, если для n условие выполнилось, продолжаю считывать данные, открываю файл для записи, вызываю функцию мистер своп, в конце записываю, что больше перестановок не потребуется, закрываю файлы. Иначе сообщая об ошибке, закрываю файл для чтения.

task_8.py	output_Task8.txt	input_Task8.txt
1 Swap elements at indices 1 and 2. ✓		1 5
2 Swap elements at indices 2 and 4.		2 3 1 4 2 2
3 Swap elements at indices 3 and 5.		
4 No more swaps needed.		

task_8.py	output_Task8.txt	input_Task8.txt
1 Swap elements at indices 2 and 3. ✓		1 3
2 No more swaps needed.		2 1 3 2

The screenshot shows a code editor with three tabs: `task_8.py`, `output_Task8.txt`, and `input_Task8.txt`. The `task_8.py` tab contains a list of swap instructions for indices 40 through 50, followed by "No more swaps needed." on line 51. The `input_Task8.txt` tab shows the input data: line 1 has "100", line 2 has "100 99 98 97 96 95 94", and line 3 is empty. The bottom panel shows the execution of `task_8.py`. It displays a table of memory usage and execution time for lines 14 through 18 of the script. The memory usage is consistently 22.3 MiB, and the execution time is 1.200009137392044e-06 seconds. The process finished with exit code 0.

```

40 Swap elements at indices 40 and 61. ✓
41 Swap elements at indices 41 and 60.
42 Swap elements at indices 42 and 59.
43 Swap elements at indices 43 and 58.
44 Swap elements at indices 44 and 57.
45 Swap elements at indices 45 and 56.
46 Swap elements at indices 46 and 55.
47 Swap elements at indices 47 and 54.
48 Swap elements at indices 48 and 53.
49 Swap elements at indices 49 and 52.
50 Swap elements at indices 50 and 51.
51 No more swaps needed.

1 100 ✓
2 100 99 98 97 96 95 94
3

Run task_8
14 22.3 MiB 0.0 MiB 2500 min = a[j]
15 22.3 MiB 0.0 MiB 2500 min_id = j
16 22.3 MiB 0.0 MiB 99 if min_id != i:
17 22.3 MiB 0.0 MiB 50 a[i], a[min_id] = a[r
18 22.3 MiB 0.0 MiB 50 f2.write(f'Swap elem

Время работы: 1.200009137392044e-06 секунд

Process finished with exit code 0

```

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	6.00004568696022e-07 секунд	22.3 MiB
Пример из задачи	5.00003807246685e-07 секунд	22.3 MiB
Верхняя граница диапазона значений входных данных из текста задачи	1.200009137392044e-06 секунд	22.3 MiB

Вывод: удивительно, но для большего количества значений данный алгоритм работает быстрее, затраты памяти не меняются.

Задача №9. Сложение двоичных чисел [N баллов]

9 задача. Сложение двоичных чисел

Рассмотрим задачу сложения двух n -битовых двоичных целых чисел, хранящихся в n -элементных массивах A и B . Сумму этих двух чисел необходимо записать в двоичной форме в $(n + 1)$ -элементный массив C . Напишите скрипт для сложения этих двух чисел.

- **Формат входного файла (input.txt).** В одной строке содержится два n -битовых двоичных числа, записанные через пробел ($1 \leq n \leq 10^3$)
- **Формат выходного файла (output.txt).** Одна строка - двоичное число, которое является суммой двух чисел из входного файла.
- Оцените асимптотическое время выполнения вашего алгоритма.

```
import time
from memory_profiler import profile

t_start = time.perf_counter()

@profile()
def binary_addition(n1, n2):
    ans = []
    n1 = n1[::-1]
    n2 = n2[::-1]
    max_size = max(len(n2), len(n1))
    n1 += (max_size - len(n1)) * [0]
    n2 += (max_size - len(n2)) * [0]
    limit = 0
    for a in zip(n1, n2):
        add = a[0] + a[1] + limit
        limit = add // 2
        ans.append(add % 2)
```

```

        if limit == 1:
            ans.append(1)

    ans = ans[::-1]
    return ''.join(map(str, ans))

if __name__ == '__main__':
    f1 = open('../txtf/input_Task9.txt', 'r')
    n1, n2 = map(str, f1.readline().split())
    n1 = [*map(int, n1)]
    n2 = [*map(int, n2)]
    f2 = open('../txtf/output_Task9.txt', 'w')
    f2.write(binary_addition(n1, n2))
    f2.close()
    f1.close()
t_start = time.perf_counter()
print("Время работы: %s секунд" %
      (time.perf_counter() - t_start))

```

```

import time
from memory_profiler import profile

```

```

t_start = time.perf_counter()

```

```

@profile()
def binary_addition(n1, n2):
    ans = []
    n1 = n1[::-1]
    n2 = n2[::-1]
    max_size = max(len(n2), len(n1))
    n1 += (max_size - len(n1)) * [0]
    n2 += (max_size - len(n2)) * [0]
    limit = 0
    for a in zip(n1, n2):
        add = a[0] + a[1] + limit
        limit = add // 2

```

```

        ans.append(add % 2)

    if limit == 1:
        ans.append(1)

    ans = ans[::-1]
    return ".join(map(str, ans))

if __name__ == '__main__':
    f1 = open('./txtf/input_Task9.txt', 'r')
    n1, n2 = map(str, f1.readline().split())
    n1 = [*map(int, n1)]
    n2 = [*map(int, n2)]
    f2 = open('./txtf/output_Task9.txt', 'w')
    f2.write(binary_addition(n1, n2))
    f2.close()
    f1.close()
t_start = time.perf_counter()
print("Время работы: %s секунд" % (time.perf_counter() - t_start))

```

Описание решения.

Создаю функцию для двоичного сложения, в нее передаю списки чисел `n1` и `n2`, которые необходимо сложить. Завожу список, в который буду записывать ответ, для удобства переворачиваю списки `n1` и `n2`.

Вычисляю максимальный размер списка, чтобы сделать их в случае различия в длине одинаковыми, исправляю путем добавления нулей.

Создаю переменную `limit`, она отвечает больше за “переполнение”. Прохожусь одновременно по `n1` и `n2`, в переменную отвечающую за сложение записываю сложение первого элемента из первого списка и первого элемента из второго плюс переполнение. Обновляю переменную переполнения, она будет целой частью от деления на 2 переменной сложения, в список ответа записываю остаток от деления сложения на 2.

Если лимит после цикла будет равен 1, то добавляю 1 и в список ответа, переворачиваю `ans`, возвращаю строку результата.

В основном коде открываю файл для считывания, считываю списки `n1` и `n2`, с помощью `map` присваиваю каждому элементу списков тип `int`, открываю файл для записи ответа. записываю туда результат выполнения функции, закрываю файлы.

The screenshots show the following input and output pairs:

- Input: 1 0 → Output: 1 1
- Input: 1 1100 110 → Output: 1 10010
- Input: 1 110011111111111111 1100000101010100101 → Output: 1 10010100101010100100

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	7.00005330145359e-07 секунд	22.4 MiB
Пример из задачи	5.00003807246685e-07 секунд	22.4 MiB
Верхняя граница диапазона значений входных данных из текста задачи	9.00006853044033e-07 секунд	22.3 MiB

Вывод: при увеличении входных данных увеличивается и время работы алгоритма, но немного уменьшается затраченная память.

Вывод

В данной лабораторной работе я ознакомилась с алгоритмами сортировки за $O(n^2)$, линейным поиском, написала функцию, которая считает сложение двоичных чисел.

Алгоритмы сортировки, представленные в работе, эффективно использовать при более маленьких значениях, потому что при больших программа может просто привести к ошибке(долго считать и тд), на мой взгляд, наиболее эффективной из этих сортировок является сортировка вставкой, тк время ее работы ниже. чем у остальных, но и она не является идеальной.

Линейный поиск также лучше всего использовать при маленьких входных данных, потому что алгоритм будет неэффективен при больших данных, в силу своей реализации, тк мы просто проходимся по элементам и сравниваем, что довольно долго будет работать при гораздо больших данных.